

CSE 256 WI26 | Lecture 1: Introduction

Ndapa Nakashole

January 6, 2026

Today

- ① A Brief History of NLP
- ② Syllabus Overview

Administrivia

- ▶ **Lecture:** Tuesdays and Thursdays at 5.00pm - 6.20pm
- ▶ **Gradescope** and **Piazza:** will be linked from Canvas
- ▶ **Office Hours:** listed on Canvas
- ▶ **Communicating with us:**
Piazza is the best way to reach us , use private posts for personal matters. (email only for truly private matters)

,

Useful Background

There are no prerequisites for the course, but useful to know:

- ▶ Probability & Linear Algebra
- ▶ PyTorch - you may find [this tutorial](#) helpful if you are new to PyTorch.

Natural Language Processing (NLP)

NLP

- ▶ NLP is the study of computer systems that take as **input** or produce as **output** natural language
 - ◉ **Input/Output:** English, Mandarin Chinese, Zulu, Spanish, ...
- ▶ Goal: give machines the power to **understand** not just words, but entire sentences, paragraphs, documents
- ▶ **Something to consider:** What does it mean to understand: surface-level appearance versus true understanding?

THE REVERSAL CURSE: LLMs TRAINED ON “A IS B” FAIL TO LEARN “B IS A”

Lukas Berglund
Vanderbilt University

Meg Tong
Independent

Max Kaufmann
UK AI Safety Institute

Mikita Balesni
Apollo Research

Asa Cooper Stickland
New York University

Tomasz Korbak
University of Sussex

Owain Evans*
University of Oxford

ABSTRACT

We expose a surprising failure of generalization in auto-regressive large language models (LLMs). If a model is trained on a sentence of the form “A is B”, it will not automatically generalize to the reverse direction “B is A”. This is the **Reversal Curse**. For instance, if a model is trained on “Valentina Tereshkova was the first woman to travel to space”, it will not automatically be able to answer the question, “Who was the first woman to travel to space?”. Moreover, the likelihood of the correct answer (“Valentina Tereshkova”) will not be higher than for a random name. Thus, models do not generalize a prevalent pattern in their training set: if “A is B” occurs, “B is A” is more likely to occur. It is worth noting, however, that if “A is B” appears *in-context*, models can deduce the reverse relationship.



Figure 1: **Inconsistent knowledge in GPT-4.** GPT-4 correctly gives the name of Tom Cruise's mother (left). Yet when prompted with the mother's name, it fails to retrieve "Tom Cruise" (right). We hypothesize this ordering effect is due to the Reversal Curse. Models trained on "A is B" (e.g. "Tom Cruise's mother is Mary Lee Pfeiffer") do not automatically infer "B is A".

Step 1 Finetune on synthetic facts shown in one order



Step 2 Evaluate in both orders

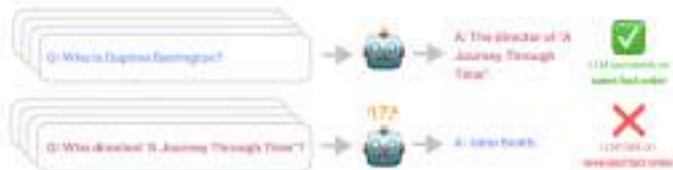


Figure 2: **Finetuning test for the Reversal Curse.** In Experiment 1, we finetune a model on fictitious facts where the name (e.g. “Daphne Barrington”) precedes the description (e.g. “the director of ...”). Then we prompt the model with questions in both orders. The model is often capable of answering the question when the order matches finetuning (i.e. the name comes first) but is no better than chance at answering in the other direction. Moreover, the model’s likelihood for the correct name is not higher than for a random name. This demonstrates the Reversal Curse.

NLP develops systems for:

- ① **Analysis** of language (NL to some useful output): e.g., text classification, question answering
- ② **Generation** of language (NL to NL; image to NL, etc): e.g., summarization, image captioning, machine translation
- ③ **Representation** of language (NL to some representation): e.g., learning word embeddings

→ In the past, systems were **task-specific**, now we have **general-purpose** systems capable of all of the above and more.

NLP Origins

NLP research has gained a lot of interest recently, but it has been around for a while, dating back to the late 1940s.

NLP Origins: Weaver's memorandum, 1949

- ▶ Generally agreed that Warren Weaver, a wartime US mathematician and scientist, brought the idea of the first computer-based application related to natural language: machine translation (MT)

Weaver in a letter to Norbert Wiener:

... one naturally wonders if the problem of translation could conceivably be treated as a problem in cryptography. When I look at an article in Russian, I say: "This is really written in English, but it has been coded in some strange symbols. I will now proceed to decode."

Weaver Memorandum inspired first MT research projects

- ▶ E.g.,: The Georgetown-IBM experiment (Dostert, 1955): the first public demonstration of MT



Fig. 2: Hurd, Dostert and Watson at the demonstration

- ◉ Demonstrated successful translation of Russian sentences translated into English
- ◉ Used hand-coded rules, dictionary look-ups, and word reordering in the target language

Language turned out to be messy and complex

- ▶ Rules, dictionary look-ups, and word reordering were not enough!
- ▶ Human languages are **ambiguous** (unlike programming and other formal languages)
- ▶ **Richness**: any meaning may be expressed many ways
- ▶ Human language interpretation depends on **real world, common sense, and contextual knowledge**
- ▶ There is **linguistic diversity** across genres, styles, ...

Get help from linguistics?

- ▶ Idea: leveraging theory of language structure and meaning could help?
- ▶ E.g., *generative grammar* (Chomsky, 1957), a rule based system of syntactic structures, brought insight into how linguistics could help MT

Even with linguistic theories ... MT was underwhelming

- ▶ MT system round-trip-translated English to Russian and back to English: come out completely off!

The spirit is willing but the flesh is weak.



(Russian)



The vodka is good but the meat is rotten.

- ▶ 1966: ALPAC cut off government funding for MT, first AI winter

Stillm, amid the AI winter(s), new theories and interesting prototypes were developed

- ▶ lates 60s - early 70s: Theoretical work focused on how to **represent meaning**
 - ◉ 1970s: **Frame semantics** (Fillmore, 1976) and **Montague semantics** (Montague, 1970) were developed

Historical Prototype: ELIZA, 1966

ELIZA (Weizenbaum, 1966) was built to replicate the conversation between a psychologist and a patient, simply by permuting or echoing the user input.

```
welcome to

      EEEEE LL      IIII ZZZZZZ AAAAA
      EE   LL      II   ZZ   AA   AA
      EEEEE LL      II   ZZZ  AAAAAA
      EE   LL      II   ZZ   AA   AA
      EEEEE LLLLLL IIII ZZZZZZ AA   AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

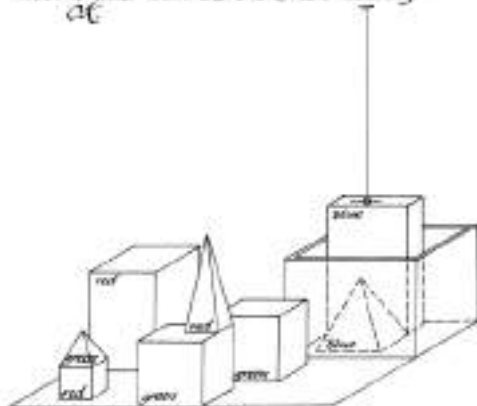
ELIZA: Is something troubling you ?
YOU:  Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:  They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:  Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:  He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:  It's true, I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

Historical Prototype: SHRDLU, Winograd 1971

- Terry Winograd's 1971 MIT thesis: using natural language to interact with a robot in a block world environment.

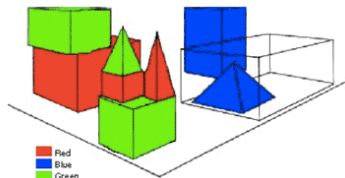
*Find a block which is taller than the one
you are holding and put it into the box.*

*By "it", I assume you mean the block
which is taller than the one I am holding
OK*



Historical Prototype: SHRDLU, 1971

- ▶ Terry Winograd's 1971 MIT thesis: using natural language to interact with a robot in a block world environment.



Person: Pick up a big red block.

Computer: OK.

Person: Grasp the pyramid.

Computer: I don't understand which pyramid you mean.

- ▶ SHRDLU inspired modern NLP research on grounded language understanding

Statistical Methods

- ▶ Statistical models revolutionized NLP in the late 1980s and early 1990s
- ▶ Early ML algorithms, eg;. decision trees matched the performance of rule-based systems.
 - ◉ But, ML was attractive because it was simpler: learned rules automatically, enabled probabilistic decision-making

Statistical Methods: IBM Models 1-5

- ▶ IBM Models 1-5 (Brown et al., 1993) were the first statistical models for machine translation

 IBM Model 1

Let f and m be the (known) lengths of e and f .
Latent variable $a = (a_1, \dots, a_m)$, each a_i ranging over $\{0, \dots, e\}$ (positions in e)
* $a_i = 0$ means that f_i is "aligned" to e_0 .

$$p(f, a | e, m) = \prod_{i=1}^m p(a_i | e, m) p(f_i | e_{a_i})$$
$$= \prod_{i=1}^m \frac{1}{e+1} t(f_i | e_{a_i}) = \left(\frac{1}{e+1} \right)^m \prod_{i=1}^m t(f_i | e_{a_i})$$

$$p(f | e, m) = \sum_{a_1=0}^e \sum_{a_2=0}^e \dots \sum_{a_m=0}^e p(f, a | e, m)$$
$$= \sum_{a \in \{0, \dots, e\}^m} p(f, a | e, m)$$

∞

In the 1990s: Transition to statistical models away from rule-based systems designed by linguists

"Everytime I fire a linguist, the performance of the speech recognizer goes up" - Fred Jelinek, IBM Research, 1990s

History of NLP: Before deep learning



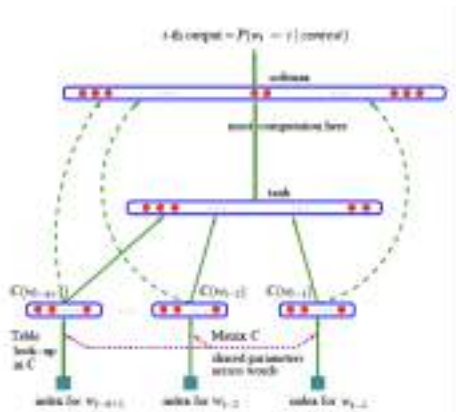
90s to early 2000s: Statistical NLP

- ▶ **Methods:** Logistic Regression, Support Vector Machines(SVM), Hidden Markov Models (HMMs), Conditional Random Fields(CRFs), etc.
- ▶ **Feature Engineering** for specific Tasks: POS tagging, Named Entity Recognition, Parsing, etc.
- ▶ **Language Models:** n-grams with smoothing

Dawn of Deep Learning Era

2003: A neural language model

- ▶ Bengio et al. (2003) proposed first neural language model: 1-hidden layer feed-forward neural network.
- ▶ Introduced word embeddings, real-valued feature vector in $\mathbb{R}^d$.



2008: Multi-task Learning “NLP Almost from Scratch”

Natural Language Processing (Almost) from Scratch

Roman Collobert*

ROMAN@ATKELLOBERT.COM

Jason Weston[†]

WESTON@GOOGLE.COM

Leon Bottou[‡]

LEON@BOTTOU.ORG

Michael Karlen

MICHAEL.KARLEN@YIHAI.COM

Koray Kavukcuoglu[§]

KORAY@CS.NYU.EDU

Pavel Kukin[¶]

PIKURA@CS.BUTTERF.SIU

AT&T Laboratories America

4 Independence Way

Princeton, NJ 08540

Editor: Michael Collins

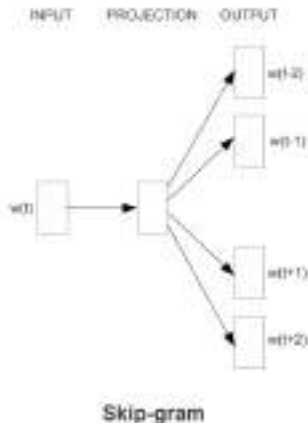
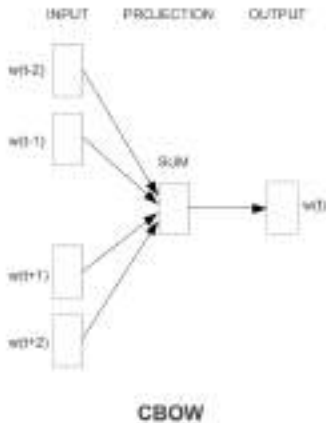
Abstract

We propose a unified neural network architecture and learning algorithm that can be applied to various natural language processing tasks including part-of-speech tagging, chunking, named entity recognition, and semantic role labeling. This versatility is achieved by trying to avoid task-specific engineering and therefore disregarding a lot of prior knowledge. Instead of exploiting man-made input features carefully optimized for each task, our system learns internal representations on the basis of vast amounts of mostly unlabeled training data. This work is fully used as a basis for building a freely available tagging system with good performance and minimal computational re-

- ▶ Entire neural network trained jointly on all the tasks using weight-sharing of the word embedding matrix.

2013: Word2Vec

In 2013, Mikolov et al. introduced arguably the most popular word embedding model: **Word2Vec**



2013-18: RNNs, LSTMs, CNNs for NLP

- ▶ Adoption of neural networks in NLP: recurrent neural networks (RNNs; Elman, 1990), later LSTMs, convolutional neural networks (CNNs), and recursive neural networks (Socher et al., 2013)
- ▶ **Architecture Engineering**: combining RNNs with CRFs for sequence labeling (2014), CNNs for text classification (2014), summarization with pointer-generators RNNs (2017) ...

2014: sequence-to-sequence

- Sequence-to-sequence learning, a general **end-to-end** approach for mapping one sequence to another using a **single neural network**. (Sutskever et al. 2014)

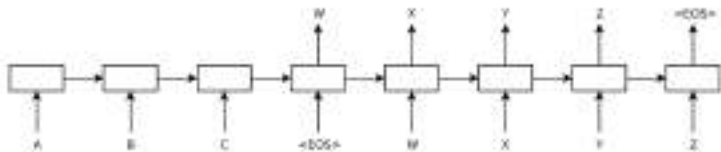
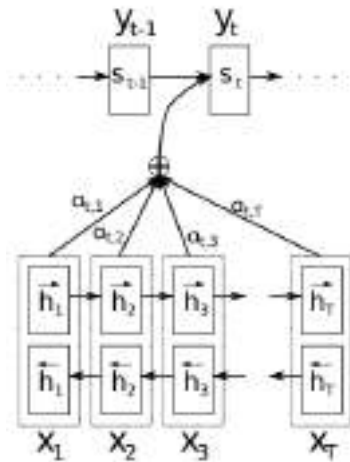


Figure 1: Our model reads an input sentence "ABC" and produces "WXYZ" as the output sentence. The model steps making predictions after outputting the end-of-sentence token. Note that the LSTM reads the input sentence in reverse, because doing so introduces many short term dependencies in the data that make the optimization problems much easier.

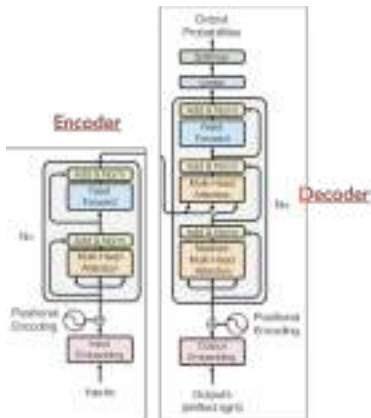
2015: Attention

- Attention to reduce the bottleneck of sequence-to-sequence models: requirement to compress the entire content of the source sequence into a fixed-size vector (Bahdanau et al. 2015)



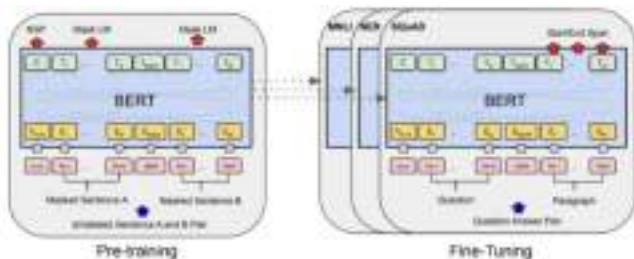
2017: Transformer

- ▶ Attention of Bahdanau still required sequential processing with RNNs; Transformer eschews recurrence, and **relies entirely on attention** mechanisms.
- ▶ Processes all parts of the data simultaneously; more efficient than RNNs and LSTMs (Vaswani et al. 2017)



(2018-) Pretrained Language Models (LLMs)

- Widespread use of pretrained LMs as an initialization for finetuning on downstream tasks due to:
 - ◉ ELMo (Peters et al., 2018), ULMFiT (Howard and Ruder, 2018), GPT (Radford et al., 2018), and BERT (Devlin et al., 2019)



Pretraining Impact (Question Answering)



Source: <https://paperswithcode.com/sota/question-answering-on-squad11-dev>

History of NLP: The Deep Learning Era



[https:](https://medium.com/@antoine.louis/a-brief-history-of-natural-language-processing-part-1-ffbc937ebce)

[//medium.com/@antoine.louis/a-brief-history-of-natural-language-processing-part-1-ffbc937ebce](https://medium.com/@antoine.louis/a-brief-history-of-natural-language-processing-part-1-ffbc937ebce)

Current State of NLP: where are we?

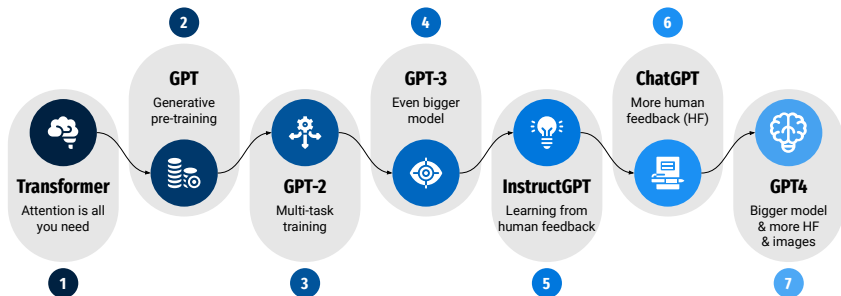
NLP systems are increasingly used in everyday life

- ▶ Search engines
- ▶ ChatGPT, and other AI assistants (generally powered by Large Language Models (LLMs))



ChatGPT: History and Trajectory

- ▶ Fastest growing consumer computing applications in history
 - ◉ Only **5 days** to get to 1M users (prior record holder Instagram: 2.5 months)

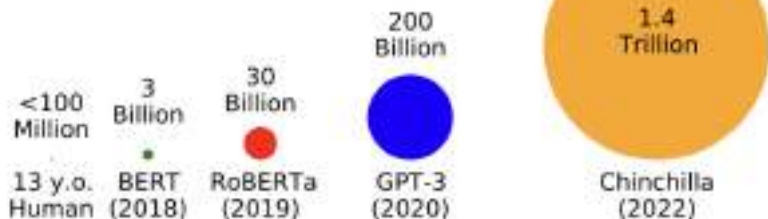


- ▶ Key ingredient: **Pretraining**

GPT: Generative Pretraining Transformer

sources: OpenAI, Statista, Reuters

Pretraining requires data, data, data!



*

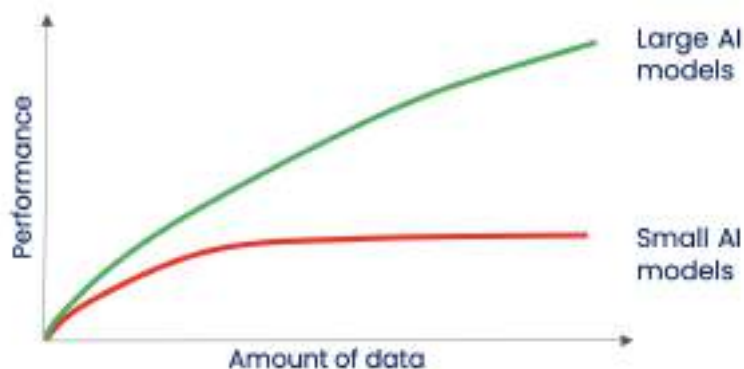
* image from: <https://babylm.github.io/>

Key advantage of language models: no humans required!

- ▶ No human annotations (vs detect cancer in chest X-rays)
- ▶ Nearly unlimited training data!



2010–2020: Large Scale Supervised Learning



* need the right **data**, the right **model**

Compute

```
import torch
import time

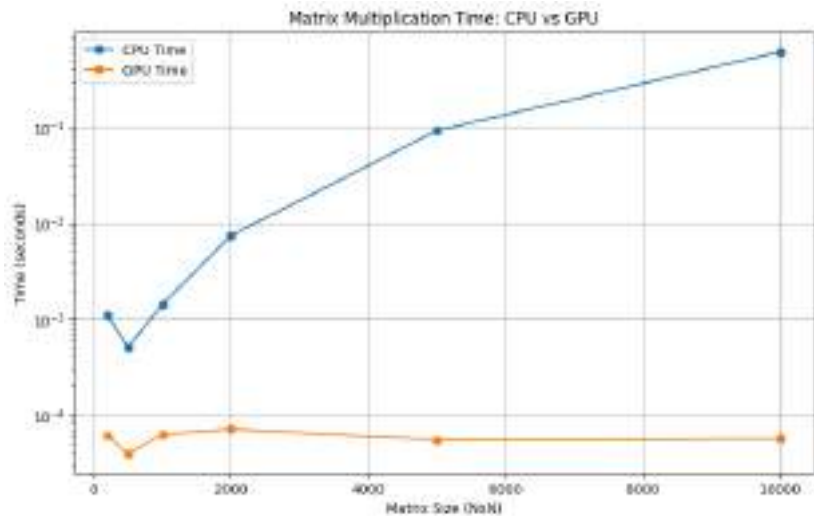
import matplotlib.pyplot as plt

def matrix_multiply(size, device):
    A = torch.randn(size, size, device=device)
    B = torch.randn(size, size, device=device)
    start_time = time.time()
    torch.matmul(A, B)
    return time.time() - start_time

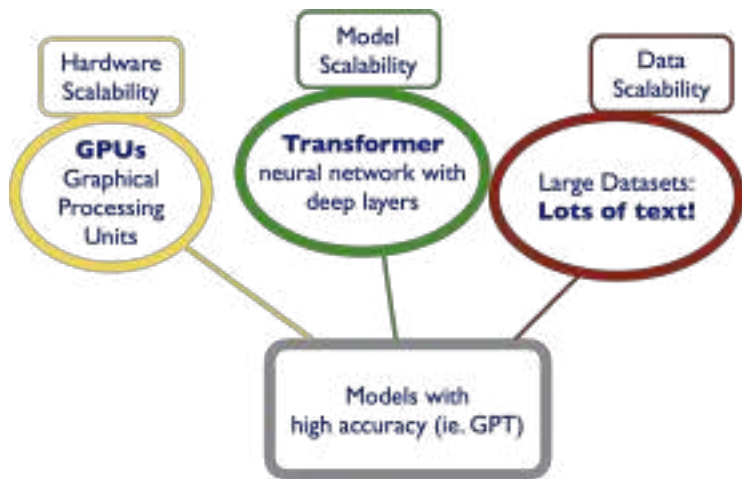
matrix_sizes = [200, 500, 1000, 2000, 5000, 10000]
cpu_device = torch.device('cpu')
gpu_device = torch.device('cuda') if torch.cuda.is_available() else None
cpu_times = []
gpu_times = []

for size in matrix_sizes:
    cpu_time = matrix_multiply(size, cpu_device)
    gpu_time = matrix_multiply(size, gpu_device) if gpu_device else float('inf')
    cpu_times.append(cpu_time)
    gpu_times.append(gpu_time)
```

Compute

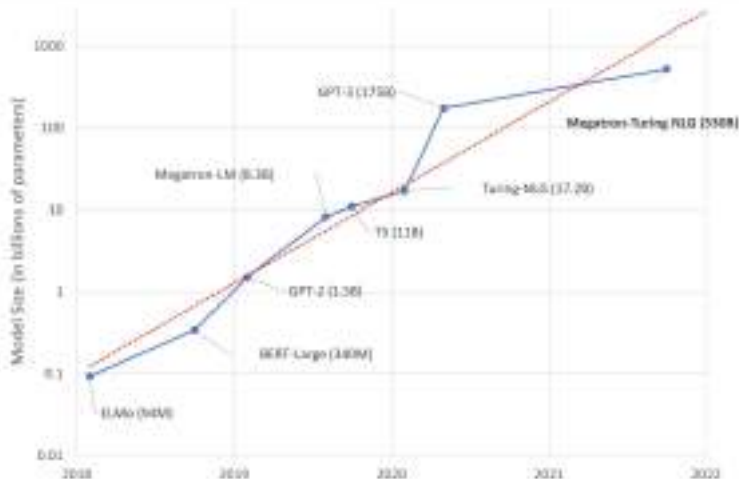


GPT: Transformer neural network + GPU + lots data



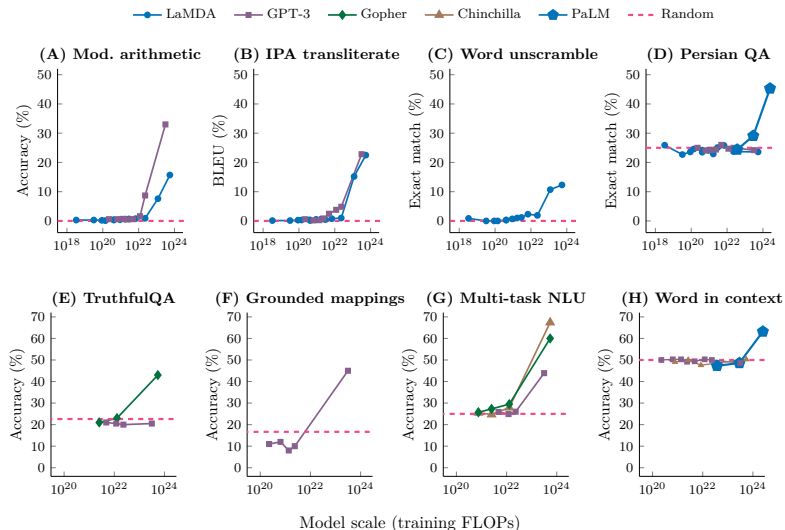
Scaling LMs: GPT → GPT2 → GPT3

- Scale is great: Just Do It!



* plot from: <https://huggingface.co/blog/large-language-models>

Scaling LMs Leads to Emergent Abilities?



*Wei, Jason, et al. "Emergent abilities of large language models." arXiv preprint arXiv:2206.07682 (2022)

Are Emergent Abilities of LLMs a Mirage?

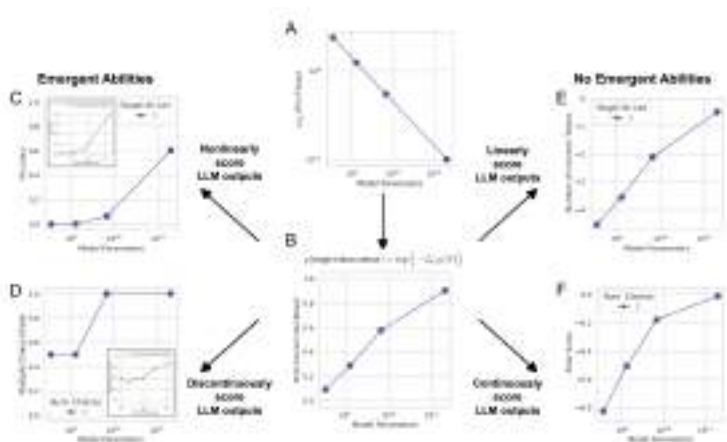


Figure 2: Emergent abilities of large language models are created by the researcher's chosen metrics, not unpredictable changes in model behavior with scale. (A) Suppose the per-token

Scale has enhanced LMs: can **Prompt** LMs to do many tasks!

- ▶ GPT-3 2020 NeurIPS paper: LM can do many tasks ... but how do we talk to them?
- ▶ **Prompt**: a cue given to the **pretrained LM** to allow it to better understand people's questions

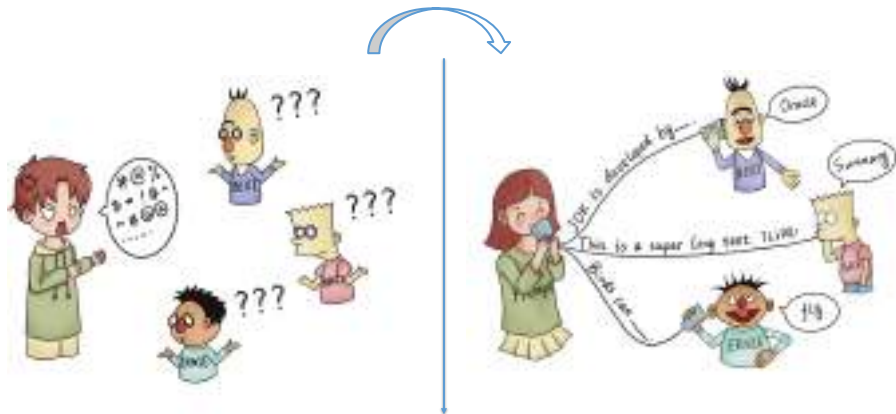


* Brown et al. Language models are few-shot learners. NeurIPS 2020;

* Liu, Pengfei, et al. "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing." ACM Computing Surveys 55.9 (2023): 1-35.

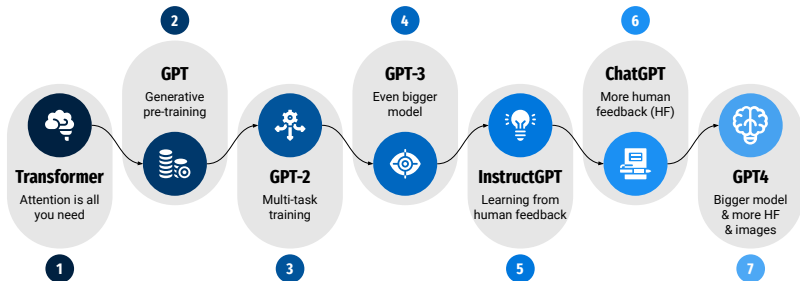
Scale has enhanced LMs: Prompting LMs to do any task!

- ▶ GPT-3 paper: LM can do any task ... but how do we talk to them?
- ▶ **Prompt:** a cue given to the **pretrained LM** to allow it to better understand people's questions



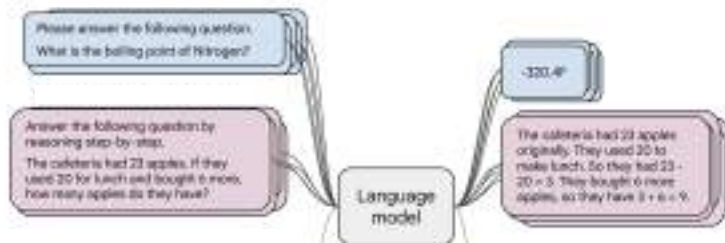
* Brown et al. Language models are few-shot learners. NeurIPS 2020;

From GPT-3 to ChatGPT



GPT-3.5: Instruction Tuning

Collect examples of (instruction, output) pairs across many tasks and finetune an LM



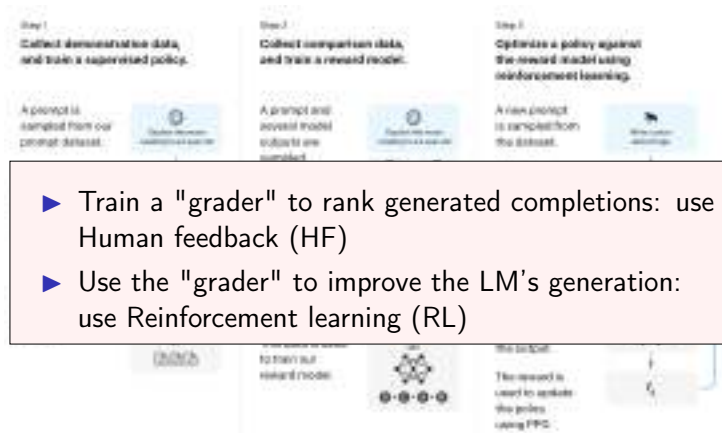
Evaluate on **unseen tasks**



[FLAN-T5; [Chung et al., 2022](#)]

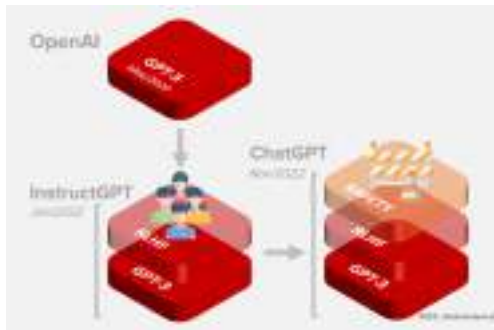
GPT-3.5: Instruction Tuning and Modeling Human Preferences via RLHF

- ▶ Reinforcement Learning from Human Feedback (**RLHF**)
- ▶ Explicitly attempt to satisfy **human preferences** using RL



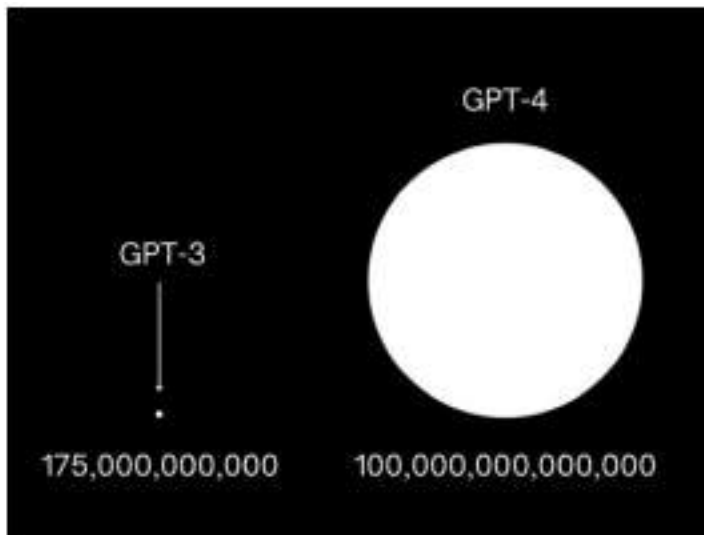
ChatGPT: A GPT-3.5 that can chat

- ▶ Fine-tuned based on chat data
- ▶ Additional safety guards → "As a LM, I cannot answer ..."
- ▶ Provides an intuitive interface for users to have a **conversation**



ChatGPT Plus: A multi-modal LM GPT-4

Image + Text; More parameters than GPT-3

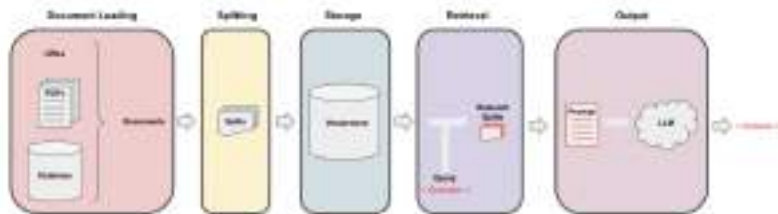


Big Idea: LLM as reasoning engine

- ▶ LLM knowledge base: large but incomplete
- ▶ Idea: retrieve information from elsewhere (i.e. private documents), then add it to the prompt and ask the LLM to process it to get an answer
- ▶ Here the LLM is used as a reasoning engine to process information, rather than using it as a source of information

Retrieval-Augmented Generation (RAG): as part of an application

► RAG pipeline



Retrieval-Augmented Generation (RAG): when using a Web user interface

- ▶ Take a piece of text and just copy it into the prompt of an online web UI of an LLM
- ▶ Instruct the LLM to use that context to generate an answer

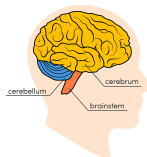
Summary of the History of Modern NLP

Big picture: Paradigms of Modern NLP Progress

- ① **Feature Engineering** (1990s - 2000s)
- ② **Architecture Engineering** (2010 - 2018, LSTMS, CNNs, Transformers)
- ③ **Objective Engineering** (2018, Pretraining: ELMo, BERT, GPT)
- ④ **Prompt Engineering** (2020s - present: prompts, instruction-tuning, chain-of-thought, etc.)

NLP vs Humans

- ▶ **General** language understanding turned out be difficult
- ▶ Some current challenges:
 - ◉ Sample efficiency: LLMs require a lot of data
 - ◉ Robustness: LLMs are brittle, can be easily fooled
 - ◉ Originality: LLMs lack ability to create truly original content, styles, ...
 - ◉ Reasoning: LLMs have limited understanding of causality, counterfactuals, multi-step logical reasoning, etc.
- ▶ **Human brain:** the only known general purpose language understanding system



Syllabus

Course Objectives

- ▶ Cover fundamental machine learning and deep learning techniques used in NLP
- ▶ Cover modern NLP problems encountered in the literature: what are the active research topics in 2025-6?
- ▶ Make you a "producer" rather than a "consumer" of NLP tools
- ▶ The assignments should teach you what you need to understand nearly any system in the literature (e.g. classification layers from PA1, Transformer backbones from PA2, etc.)

Course Content

- ① **Intro to Neural NLP:** linear models, feed forward neural networks, word embeddings (2 weeks)
- ② **Modern Language Models (LMs): Background, key ingredients** (3.5 weeks)
- ③ **Modern LMs: In practice:** parameter efficient fine-tuning, question answering, LMs for programming, interpretability, Ethics (4.5 weeks)

Readings

The readings are assigned to complement the lectures. Readings can be consulted before lecture as preparation or after lecture to review.

- ▶ **[Goldberg]**: A Primer on Neural Network Models for Natural Language Processing
- ▶ **[Eisenstein]**: Natural Language Processing
- ▶ **[ZLLS]**: Dive into Deep Learning
- ▶ **[J&M]**: Speech and Language Processing
- ▶ Research papers

Grades

► Deliverables:

① **Programming Assignments** (62% - 20/20/22).

- **code no-show policy**, learning to debug your own code is a critical skill. Code no-show applies to TA, instructor, classmates, and others.

② **Midterm exam:** (38%). March 3, make sure to be there, there is no makeup exam.

► Computing Resources:

- ◉ Google Colab is a nice resource (especially Colab Pro, \$9.99/mo) Unfortunately, we cannot provide compute credits

► Late Policy:

- ◉ Assignments submitted within 24 hours after the deadline incur a 15% penalty; submissions within 48 hours incur a 30% penalty. No submissions are accepted after 48 hours.

Grades, important notes

- ① No additional chances for earning extra credit, make sure to turn in all assignments.
- ② Grades are not rounded or curved
- ③ **Academic Integrity:** All assignments are individual, no group work.

Use of AI tools

You can use AI tools such as ChatGPT or Copilot to help you write or code. If you do, disclose this in your report and specify:

- ▶ The degree to which you changed the outputs of the tool (no changes, minor changes, or major changes)

Grading Scale

Letter Grade	Percentage
A	93+
A-	90+
B+	87+
B	83+
B-	80+
C+	77+
C	73+
C-	70+
D	60+
F	below 60

** A+: given at the instructor's discretion.

*** P/NP: If you are taking the course P/NP, you will receive a grade of P if you meet the criteria for a B- grade (on the midterm), otherwise you will receive a grade of NP.

Enrollment Questions

- ▶ **Q:** I am on the waitlist, but I really want to get into the course. Can you let me in?
- ▶ **A:** Unfortunately, instructors are not able to enroll students in classes. Please direct your questions about enrollment to CSE course support staff. You can send an email to: Tina Chung - tichung@ucsd.edu
- ▶ **Q:** What are my chances of getting off the waitlist?
- ▶ **A:** The instructional staff is not equipped to answer this question. Drop a line to: Tina Chung - tichung@ucsd.edu

That's all for today!

► Questions?

The content is inspired by many sources, and people.
Attribution might be lacking where it should be

Instructor: Ndapa Nakashole

Office Hours: Tuesday 11.00am -12.00 pm in CSE 4109

Lectures: TuTh 5pm-6.20pm in PETER 108

Slides: posted under "Syllabus"

TAs:

TA/tutor Name	Office hours	Room
Garg, Devanshi (d2garg@ucsd.edu)	Friday 4 pm - 5pm	B250A
Ranka, Sharanya Milind (smranka@ucsd.edu)	Tuesday 1pm - 2pm	B275
Wajanasara, Pisit (pwajanasara@ucsd.edu)	Monday 9am - 10am	B250A
Wang, Ruiyi (ruw079@ucsd.edu)	Friday 11am - 12pm	CSE 4242

Course Description

This course introduces Natural Language Processing (NLP), focusing on modern deep learning methods and foundation models for language. Topics include neural and transformer architectures, pre-trained and instruction-tuned large language models, parameter-efficient fine-tuning (PEFT), prompting, and post-training. Core problems emphasize semantic understanding and generation (e.g., question answering, and machine translation). Programming assignments involve building, and evaluating contemporary NLP systems.

Readings. The readings are assigned to complement the lectures. Readings can be consulted before lecture as preparation or after lecture to review.

Book 1: **[Goldberg]** [Goldberg: A Primer on Neural Network Models for Natural Language Processing](https://arxiv.org/pdf/1510.00726.pdf) .

Book 2: **[Eisenstein]** [Eisenstein: Natural Language Processing](https://github.com/jacobeisenstein/gt-nlp-class/blob/master/notes/eisenstein-nlp-notes.pdf) 
<https://github.com/jacobeisenstein/gt-nlp-class/blob/master/notes/eisenstein-nlp-notes.pdf>

Book 3: **[ZLLS]** [Aston Zhang, Zack C. Lipton, Mu Li, Alex J. Smola, Dive into Deep Learning](https://d2l.ai/index.html) .

Book 4: [J&M] [Jurafsky and Martin: Speech and Language Processing \(3rd ed. draft\)](https://web.stanford.edu/~jurafsky/slp3/) 
[\(https://web.stanford.edu/~jurafsky/slp3/\)](https://web.stanford.edu/~jurafsky/slp3/)

Late Submission Policy. Assignments submitted within 24 hours after the deadline incur a 15% penalty; submissions within 48 hours incur a 30% penalty. No submissions are accepted after 48 hours.

Grade:

Programming Assignments **(62%)**

Midterm **(38%)**

Letter Grade	Percentage
A	93+
A-	90+
B+	87+
B	83+
B-	80+
C+	77+
C	73+
C-	70+
D	60+
F	below 60

- **A+:** given at the instructor's discretion.

- **P/NP:** If you are taking the course P/NP, you will receive a grade of P if you meet the criteria for a B-grade on the Midterm, otherwise you will receive a grade of NP.

All coursework is to be carried out individually, there are no team project or assignments.

Computing Resources:

- Assignments can run on a modern personal computer or laptop
- Alternative: [Google Colab](https://colab.research.google.com/?utm_source=scs-index)  (https://colab.research.google.com/?utm_source=scs-index)

Academic Integrity, AI tools, etc:

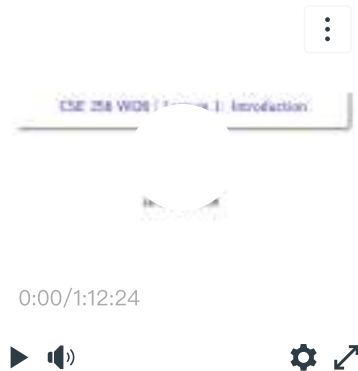
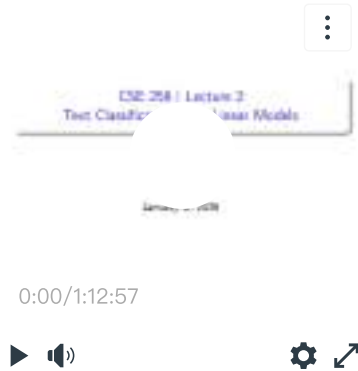
You can use AI tools such as ChatGPT or Copilot to help you write or code. If you do, disclose this in your report and specify:

- The degree to which you changed the outputs of the tool (no changes, minor changes, or major changes)

It is important to comprehensively understand academy integrity rules, which apply to all your submitted work, as set by the university: <https://academicintegrity.ucsd.edu/>
[\(https://academicintegrity.ucsd.edu/\)](https://academicintegrity.ucsd.edu/)

Course Syllabus

Tentative Schedule: Minor changes may be made as needed throughout the quarter.

Lecture #	Title	Readings
wk1: Lecture 1: Jan 6 Course Intro	Course overview & logistics (https://canvas.ucsd.edu/courses/71784/files/16957408?wrap=1)  (https://canvas.ucsd.edu/courses/71784/files/16957408/download?download_frd=1)  0:00/1:12:24	C. Manning: Linguistics & Deep Learning  (https://aclanthology.org/J4006.pdf) PyTorch Tutorial from Stanford CS224N:  (https://colab.research.google.com/drive/1AiBuEL_V9u16jGMctpYN3Byx76usp=sharing)
wk1: Lecture 2: Jan 8 Intro to Neural NLP	Machine Learning foundations: linear models (https://canvas.ucsd.edu/courses/71784/files/16973407?wrap=1)  (https://canvas.ucsd.edu/courses/71784/files/16973407/download?download_frd=1)  0:00/1:12:57 Note: There are sound issues with this recording starting around 35 minutes. We are looking into it.	[Eisenstein] 2.1; 2.5.0 - 2.5.1 A Few Useful Things to Know About Machine Learning  (https://homes.cs.washington.edu/~pedrod/papers/cacm12.pdf)
wk2: Lecture 3: Jan 13 Intro to Neural NLP	ML Foundations: Loss Functions	A Few Useful Things to Know About Machine Learning  (https://homes.cs.washington.edu/~pedrod/papers/cacm12.pdf)

wk2: Lecture 4: Jan 15 Intro to Neural NLP	ML Foundations: Optimization	• A Few Useful Things to Know About Machine Learning ↗ (https://homes.cs.washington.edu/~pedrod/papers/cacm12.pdf)
wk3: Lecture 5: Jan 20 Intro to Neural NL	Neural Networks foundations: feedforward neural networks	• [Eisenstein] 3.0-3.3 • Deep Averaging Networks ↗ (https://aclanthology.org/P15-1162.pdf) • Natural Language Processing with Small Feed-Forward Networks ↗ (https://aclanthology.org/D17-1309.pdf)
wk3: Lecture 6: Jan 22 Intro to Neural NLP	Word Embeddings & Tokenization	• Mikolov et al. 2013 word2vec ↗ (https://arxiv.org/pdf/1301.3781.pdf) • Pennington et al. 2014 GloVe ↗ (https://nlp.stanford.edu/pubs/glove.pdf) • Tokenization: Sennrich et al. 2016, Byte Pair Encoding ↗ (https://aclanthology.org/P16-1162.pdf)
wk4: Lecture 7: Modern LMs: Key ingredient	Self-Attention Attention & Transformers	Attention and Transformers Vaswani et al. 2017 (https://arxiv.org/pdf/1706.03762.pdf) Karpathy 2023, Let's build GPT (https://www.youtube.com/watch?v=kCc8FmEb1nY&ab_channel=AndrejKarpathy) Illustrated Transformer (https://jalammar.github.io/illustrated-transformer/) Beltagy et al. 2020 (https://arxiv.org/pdf/2004.05150.pdf)
wk4: Lecture 8: Modern LMs: Key ingredients	Pretraining part 1: Encoders (BERT/ELECTRA/DeBERTa)	Peters et al. 2018 ELMo (https://arxiv.org/pdf/1802.05365.pdf) Devlin et al 2019 BERT (https://arxiv.org/abs/1810.04805) Clark et al 2020 ELECTRA (https://arxiv.org/pdf/2003.10555.pdf) He 2021 DeBERTa (https://arxiv.org/pdf/2006.03654.pdf)
wk5: Lecture 9: Modern LMs: Key ingredients	Pretraining part 2: Decoders (GPT); Decoding Methods; Scaling Laws	GPT2 ↗ (https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf) Holtzman 2019 Nucleus Sampling (https://arxiv.org/pdf/1904.09751.pdf)
wk5: Lecture 10: Modern LMs: Key ingredients	Prompting; Instruction Tuning	Brown et al. 2020 GPT3 (https://arxiv.org/pdf/2005.14165.pdf) Min et al. 2022 Rethinking Demonstrations ↗ (https://arxiv.org/pdf/2202.12837.pdf) Sanh et al 2022 Instruction Tuning ↗ (https://arxiv.org/pdf/2110.08207) FLAN-T5 ↗ (http://Scaling%20Instruction-Finetuned%20Language%20Model)
wk6: Lecture 11:	Retrieval Augmented Generation (RAG)	RAG:

Modern LMs: Key ingredients		Lewis et al. 2020 RAG  https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7Paper.pdf Karpukhin et al. 2020 DPR (https://aclanthology.org/2020.emnlp-main.550.pdf) Borgeaud et al. 2022 RETRO (https://arxiv.org/abs/2112.04426)
wk6: Lecture 12: November 7 Modern LMs: In practice		
wk7: Lecture 13:		
wk7: Lecture 14: Modern LMs: In practice	Parameter Efficient Fine-tuning (PEFT) PEFT part 2 (Lottery Tickets/LoRA/Adapters)	https://aclanthology.org/2022.acl-long.125.pdf Frankle & Cabin 2019, Lottery Ticket Hypothesis https://arxiv.org/pdf/1803.03635 Ansell et al 2022, Sparse-finetuning (https://aclanthology.org/2022.acl-long.125.pdf) https://aclanthology.org/2022.acl-long.125.pdf
wk8: Lecture 15: Modern LMs: In practice	Interpretability of Neural NLP (Probes/Sparse Autoencoders/Dataset Artifacts)	https://arxiv.org/pdf/1611.01603 Huben et al, ICLR 2024  (https://openreview.net/forum?id=F76bwRSLeK) Levy et al, ACL 2024  (https://aclanthology.org/2024.acl-long.818.pdf) Conneau et al, 2018 (https://aclanthology.org/P18-1198.pdf) https://aclanthology.org/2021.emnlp-main.446.pdf Belinkov 2020 (https://aclanthology.org/2022.cl-1.7.pdf) https://arxiv.org/pdf/1611.01603
wk8: Lecture 16: Modern LMs: In practice	Applications: Application 1: Code Generation Application 2: Question Answering	Code gen: Chen et al 2021 Codex (https://arxiv.org/pdf/2107.03374) https://arxiv.org/pdf/1611.01603 https://arxiv.org/pdf/1611.01603 https://arxiv.org/pdf/1611.01603 Li et al 2022 AlphaCode (https://arxiv.org/pdf/2203.07814) https://arxiv.org/pdf/1611.01603 https://arxiv.org/pdf/1611.01603 https://arxiv.org/pdf/1611.01603 Ahn et al 2022 SayCan (https://say-can.github.io/assets/palm_saycan.pdf) QA: Chen et al 2017 DrQA (https://arxiv.org/pdf/1704.00051) Seo et al 2019 BiDAF (https://arxiv.org/pdf/1611.01603)

wk9: Lecture 17: March 3 Midterm	Midterm	https://aclanthology.org/P19-1357.pdf https://arxiv.org/pdf/1611.01603
wk9: Lecture 18: November 28	LLMs & Society • Privacy & LLMs • Bias/Incivility/Unethical use • Safeguarding LLMs	Carlini et al, 2021: extracting data from LLMs  https://www.usenix.org/system/files/sec21-carlini-extracting.pdf Jurgens et al 2018 (https://aclanthology.org/P19-1357.pdf) Bender et al 2021, stochastic parrots https://dl.acm.org/doi/10.1145/3442188.3445922
wk10: Lecture 19: Modern LLMs: In practice	• LLMs practical considerations • Open Problems in NLP • Beyond English • Course Wrap-Up	
wk10: Lecture 20: Modern LLMs: In practice		

Readings. The assigned readings are intended to complement the lectures. You may consult them before the lecture to prepare or after the lecture to review the material. The majority of our readings will be technical papers, as we do not have a designated textbook. However, for those interested in additional references, the following books are recommended:

Book 1: [Goldberg] [Goldberg: A Primer on Neural Network Models for Natural Language Processing](#)  (<https://u.cs.biu.ac.il/~yogo/nnlp.pdf>).

Book 2: [Eisenstein] [Eisenstein: Natural Language Processing](#) (<https://canvas.ucsd.edu/courses/71784/files/16887469?wrap=1>). 
https://canvas.ucsd.edu/courses/71784/files/16887469/download?download_frd=1

Book 3: [ZLLS] [Aston Zhang, Zack C. Lipton, Mu Li, Alex J. Smola, Dive into Deep Learning](#)  (<https://d2l.ai/index.html>)

Book 4: [J&M] [Jurafsky and Martin: Speech and Language Processing \(3rd ed. draft\)](#)  (<https://web.stanford.edu/~jurafsky/slp3/>)

Course Summary:

Date	Details	Due
Tue Jan 20, 2026	 The #FinAid Quizz (https://canvas.ucsd.edu/courses/71784/assignments/1082830)	due by 11:59pm
Fri Jan 30, 2026	 PA1 (https://canvas.ucsd.edu/courses/71784/assignments/1085253)	due by 11:59pm